Name: Cohort/Batch: Date: Score:

EMBEDDED C/C++ PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A IoT & Edge

TOTAL: 10 QUESTIONS

Q1 What is Embedded C/C++ and what problem in IoT & Edge does it solve? — Threat Model

Q2 What are the core components or concepts in Embedded C/C++? — Architecture

Q3 How does Embedded C/C++ compare to its main alternatives? — Configuration

Q4 How do you get started with Embedded C/C++ for a new project? — Hardening

Q5 What configuration options most affect Embedded C/C++'s behavior in production? — SSO / IdP

Q6 How does Embedded C/C++ handle reliability and high availability? — Observability

Q7 What observability does Embedded C/C++ provide out of the box? — Lifecycle

Q8 What are common performance bottlenecks in Embedded C/C++? — Compliance

Q9 What security best practices apply when running Embedded C/C++? — Testing

Q10 What mistakes do teams commonly make when adopting Embedded C/C++? — Tuning

ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Embedded C/C++ is a tool or

Mitigation

Embedded C/C++ is a tool or platform that automates or simplifies a specific concern in the IoT & Edge domain, reducing manual effort and operational complexity for engineering teams.

A2 Embedded C/C++ has key building blocks

Primitives

Embedded C/C++ has key building blocks that work together: a configuration layer defines intent, a runtime layer enforces it, and an API or CLI layer allows operators to manage and observe the system.

A3 Embedded C/C++ trades off simplicity against

Hardening

Embedded C/C++ trades off simplicity against feature richness differently than its alternatives. Choose Embedded C/C++ when its specific strengths performance, ecosystem, or ease of use align with your team's primary constraints.

A4 Install Embedded C/C++ via its package

Gotchas

Install Embedded C/C++ via its package manager or binary release. Follow the quickstart to initialize a project, configure the minimal required settings, and run the default command to verify the setup works end-to-end.

A5 Production-critical settings typically include

Federation

Production-critical settings typically include concurrency limits, timeout values, retry policies, resource limits, and logging levels. Review the official production hardening guide before deploying.

A6 Embedded C/C++ provides HA through leader

Observability

Embedded C/C++ provides HA through leader election, replication, or stateless horizontal scaling. Deploy at least two instances across availability zones and configure health checks for automatic failover.

A7 Embedded C/C++ exposes metrics (Prometheus

Automation

Embedded C/C++ exposes metrics (Prometheus endpoint or built-in dashboard), structured logs, and optionally distributed traces. Define SLOs on the key metrics and alert before users are affected.

A8 Performance bottlenecks typically stem from

Governance

Performance bottlenecks typically stem from misconfigured connection pools, large payload sizes, insufficient worker threads, or missing caches. Profile with built-in diagnostics before optimizing.

A9 Run Embedded C/C++ with the principle

Assessment

Run Embedded C/C++ with the principle of least privilege, enable TLS for all communication, use a secrets manager for credentials, and keep the software patched to the latest stable release.

A10 Common pitfalls include skipping the

Optimization

Common pitfalls include skipping the documentation and learning the API by trial and error, using default configurations in production, lacking a runbook for operational issues, and not testing failover scenarios.